

Control y Programación de Robots

Simulación y control de robot submarino

Curso 2025/2026



Alejandro Noci Morales
Carmen Muñoz Carmona
Alejandro Roche Aniento
Eduardo Espinosa Ponce
Francisco Delgado Capote

Índice

1	Introducción	3
1.1	Contexto y motivación	3
1.2	Objetivos y metodología	3
1.3	Herramientas	3
1.4	Búsqueda del repositorio y elección	4
2	Modelo de Robot Submarino	6
2.1	Modelado Cinemático	6
2.2	Modelo Dinámico	8
2.3	Modelo Robot proporcionado	9
3	Diseño del Sistema de Control	11
3.1	Desvinculación del AUV	11
3.2	Arquitectura del Nodo de Control	11
3.3	Estrategias de Control	12
3.4	Estrategias de Control Implementadas	14
4	Pruebas y Resultados	17
4.1	1ª Prueba: Validación de Actuadores	17
4.2	2ª Prueba: Telemetría y Sensores	17
4.3	3ª Prueba: Cambio de Referencia Dinámico (Teleoperación)	17
4.4	4ª Prueba: Implementación de perturbaciones	18
4.5	5ª Prueba: Implementación y Validación del Nodo de Guiado Autónomo	19
5	Conclusiones	20
A	Guía de instalación de Orca4	21
A.1	Paso 1: Añadir repositorios adicionales (APT)	21
A.2	Paso 2: Compilar el piloto automático (ArduSub SITL)	21
A.3	Paso 3: Instalación de dependencias de compilación	21
A.4	Paso 4: Preparación del workspace (<code>colcon_ws</code>)	22
A.5	Paso 5: Instalación de dependencias de ROS y MAVROS	22
A.6	Paso 6: Compilación final	22
A.7	Paso 7: Comprobación de la instalación	22
B	Guía de puesta en marcha	23
B.1	Ejecución del Entorno	23
B.2	Compilación y Modificaciones	23
B.3	Selección del Controlador	23
B.4	Gestión de Referencias y Alto Nivel	24
B.5	Pruebas de Robustez (Corrientes)	24

1. Introducción

1.1. Contexto y motivación

Los robots submarinos se están empezando a desarrollar actualmente, lo que está suponiendo la vanguardia de la tecnología oceánica. Los Vehículos Submarinos Autónomos, también conocidos como AUVs, operan en entornos complejos donde mantener el control supone todo un reto. La no linealidad está a la orden del día en el mar, por lo que supone un desafío significativo poder actuar sobre estos vehículos.

Aun así, siendo escasa la información disponible y numerosas las adversidades, decidimos embarcarnos en este proyecto con el objetivo de aprender y desarrollar diferentes controladores para AUVs. Otra motivación para escoger esta temática ha sido el deseo de trabajar en un ámbito distinto al abordado habitualmente en clase, saliendo de la zona de confort a la que estamos acostumbrados.

Con este trabajo hemos querido poner en práctica todos los conocimientos adquiridos en la rama de la robótica y comprobar hasta dónde somos capaces de llegar.

1.2. Objetivos y metodología

El objetivo principal de este proyecto ha sido la programación de un controlador eficaz a bajo nivel para un robot submarino, cuya finalidad es mantener la estabilidad del sistema frente a las adversidades del entorno acuático.

Los movimientos que el robot es capaz de realizar incluyen descender hasta una determinada profundidad, mantenerse estable en ella mientras se desplaza hacia distintos puntos del mapa y, posteriormente, ascender de nuevo hasta la superficie sin perder el equilibrio en ningún momento.

En cuanto a la metodología seguida, el primer paso consistió en el diseño de un controlador PID sin tener en cuenta la dinámica del sistema. Posteriormente, se incorporó la dinámica del modelo al controlador para mejorar su comportamiento.

El siguiente paso fue la implementación de un controlador no lineal, concretamente un controlador SDRE, que constituye una extensión del controlador LQR. Finalmente, se programó un controlador de alto nivel de tipo punto a punto, con el objetivo de demostrar que la estabilidad del robot se mantiene ante cualquier situación a la que deba enfrentarse.

1.3. Herramientas

La plataforma de simulación empleada en este proyecto proviene del repositorio Orca4, el cual está implementado para ROS 2 y utiliza Gazebo Harmonic como entorno de simulación. Además, se emplea RViz para visualizar la posición del robot y enviarle puntos objetivo.

Este repositorio proporciona las funcionalidades básicas de un AUV, concretamente del modelo de robot submarino *BlueRobotics BlueROV2*. Asimismo, está conectado a un puente denominado MAVROS, que permite la comunicación entre el robot y los controladores de alto y bajo nivel, siendo este último ArduSub. Posteriormente se comentará mejor todo lo disponible en el repositorio.

Para la realización y prueba de nuestros propios prototipos, se desvinculó el puente MAVROS y los controladores asociados al modelo AUV. Esto dio lugar a la creación de un puente propio mediante `ros_gz_bridge`, a través del cual se conectan los programas desarrollados con el robot para permitir su movimiento en el entorno acuático.

El lenguaje de programación empleado durante todo el proyecto ha sido Python. Además, se han utilizado archivos `.launch` para ejecutar simultáneamente todos los nodos necesarios para la simulación del robot mediante un único comando.

1.4. Búsqueda del repositorio y elección

Inicialmente se han estudiado diversos repositorios, paquetes y entornos que pudieran ser útiles a la hora de lograr el control a bajo nivel. Entre los valorados se encuentran:

- **eeUVsim_Gazebo** [1]
 - Simulación de vehículo(s) submarino(s) (AUV) usando Gazebo y ROS 2
 - Soporte para entrenamiento de agentes de aprendizaje por refuerzo (RL)
 - Dinámica basada en el modelo de Thor I. Fossen (6 DOF)
 - Usa eeUVsim_Gazebo como simulador
- **UUV Simulator** [2]
 - Unmanned Underwater Vehicle (UUV)
 - Gazebo
 - Implementación Fossen
 - Plugin de sensores
 - Simulación de aletas
 - Algoritmo de asignación de esfuerzos basado en CasADi (librería de optimización numérica)
- **Stonefish** [3]
 - Librería en C++
 - Computación de hidrodinámica avanzada
 - Entornos acuáticos realistas
- **Blue** [4]
 - Entorno de un solo robot
 - Desarrollo, tests y uso de robots submarinos
 - Modelado y configuraciones en Gazebo de robots
 - Controladores AUV específicos para el robot modelado
 - Implementación de localización marina
- **Dave** [5]
 - Usa ROS1 (en migración a ROS2)
 - Entorno de simulación con ROS y Gazebo
 - Incluye sensores: sonar, Doppler velocity logs, Lidar
 - Varios vehículos diferentes
 - Plugins de corrientes, coordenadas esféricas, detector de presión, nivel de profundidad, cámara, etc.
- **Plankton** [6]
 - Simulador para investigadores de robots submarinos
 - Usa ROS2
 - Simuladores cercanos a la realidad
 - Núcleo abierto

- Simula entorno real: corrientes, oleaje, cambios de densidad, etc.
- Basado en UUV Simulator y Gazebo
- Sensores: IMU, sonar, barómetro

■ **Orca4** [7]

- Usa el robot de Blue con distintos paquetes en ROS2
- Usa ArdaSub para los controladores
- GCS (estación de control terrestre) para control y supervisión del UUV
- Usa Gazebo
- Ofrece barómetro para profundidad e IMU para orientación
- Incluye cámara opcional

Orca4 se ha seleccionado como plataforma principal para el desarrollo de este proyecto debido a su elevado grado de realismo, integración y adecuación a los objetivos planteados. En primer lugar, utiliza el modelo del robot submarino Blue, el cual cuenta con una versión física real, lo que permite trabajar con un sistema cuya geometría, sensores y comportamiento están alineados con un robot existente, aumentando así la validez del entorno de simulación.

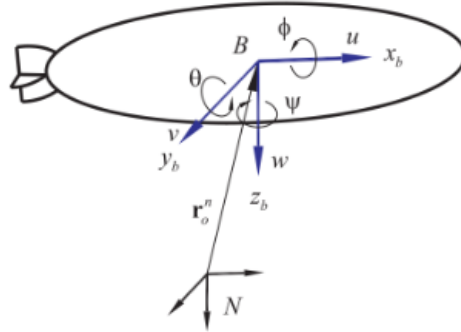
Además, Orca4 emplea ArdaSub para el control a bajo nivel, proporcionando una arquitectura de control realista y bien estructurada sobre la que desarrollar y evaluar los controladores diseñados en este trabajo. Sin embargo, esta parte se buscará desacoplarla para poder introducir con un puente los controles a desarrollar como se ha comentado previamente.

Por último, el entorno incluye sensores relevantes como barómetro, IMU y cámara, así como una estación de control terrestre (GCS) para la supervisión del vehículo.

En conjunto, estas características convierten a Orca4 en una plataforma robusta, realista y adecuada para el desarrollo de los objetivos del proyecto.

2. Modelo de Robot Submarino

2.1. Modelado Cinemático



Para describir la posición, orientación y movimiento de un vehículo submarino (robot) se utilizan dos sistemas de referencia principales:

■ **Marco de referencia inercial NED:**

- Ubicado sobre la superficie de la Tierra.
- Siglas en inglés *North-East-Down* (Norte-Este-Abajo).
- Ejes:
 - x : apunta hacia el norte
 - y : apunta hacia el este
 - z : apunta hacia abajo, normal a la superficie terrestre
- El plano x - y es tangente a la superficie de la Tierra.
- En este trabajo se denomina simplemente como marco N .

■ **Marco de referencia del cuerpo (B):**

- Fijo al cuerpo del robot (Body).
- El origen O se ubica generalmente en el centro de gravedad o centro de flotabilidad.
- Los ejes se eligen según los ejes principales de inercia:
 - x_b : eje longitudinal (de popa a proa)
 - y_b : eje transversal (de babor a estribor)
 - z_b : eje vertical (hacia abajo)

2.1.1. Variables de movimiento

Las variables de movimiento (velocidades lineales y angulares) se definen con respecto al marco del cuerpo B :

- **Ángulos de Euler:** (ϕ, θ, ψ) , que transforman coordenadas del marco N al marco B .
- **Velocidades angulares:** (p, q, r) , derivadas temporales de los ángulos de Euler.

Movimiento	Nombre		Posición	Velocidad	Fuerza
	Español	Inglés			
Traslación en x	Avance	<i>Surge</i>	x	u	X
Traslación en y	Desvío	<i>Sway</i>	y	v	Y
Traslación en z	M. Vertical	<i>Heave</i>	z	w	Z
Rotación en x	Alabeo	<i>Roll</i>	ϕ	p	K
Rotación en y	Cabeceo	<i>Pitch</i>	θ	q	M
Rotación en z	Guiñada	<i>Yaw</i>	ψ	r	N

Tabla 1: Movimientos, variables cinemáticas y fuerzas asociadas

2.1.2. Representación de posición, velocidad y fuerzas

- **Posición:** Vector del origen del cuerpo B respecto al marco inercial N se tiene el vector:

$$\eta = \begin{bmatrix} \mathbf{r}_0^n \\ \Theta \end{bmatrix}, \quad \mathbf{r}_0^n = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Θ : vector que define la orientación relativa entre los marcos N y B .

- **Velocidad del vehículo:**

$$\mathbf{v} = \begin{bmatrix} \mathbf{v}_b^o \\ \boldsymbol{\omega}_b^b \end{bmatrix}, \quad \mathbf{v}_b^o = \begin{bmatrix} u \\ v \\ w \end{bmatrix}, \quad \boldsymbol{\omega}_b^b = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

$\mathbf{v}_b^o$ es la velocidad lineal del vehículo medido en coordenadas del marco del cuerpo B, mientras que $\boldsymbol{\omega}_b^b$ representa la velocidad angular del cuerpo con respecto al marco inercial medido en el marco de referencia del cuerpo B.

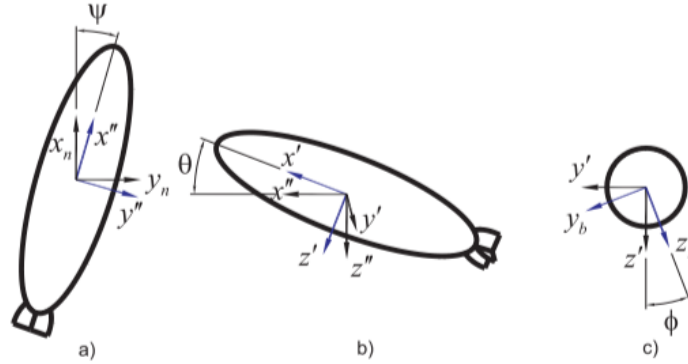
- **Fuerzas y momentos:**

$$\boldsymbol{\tau} = \begin{bmatrix} \mathbf{f}_o^b \\ \mathbf{n}_o^b \end{bmatrix}, \quad \mathbf{f}_o^b = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \quad \mathbf{n}_o^b = \begin{bmatrix} K \\ M \\ N \end{bmatrix}$$

$\mathbf{f}_o^b$ y $\mathbf{n}_o^b$ son los vectores de fuerza y momento ejercidos sobre el cuerpo medidos en el marco de referencia del cuerpo B.

2.1.3. Ángulos de Euler y matriz de rotación

Secuencia de rotaciones de los ángulos de Euler:



- Rotación del marco de referencia de un ángulo ψ alrededor del eje z para obtener el marco B .
- Rotación del marco de referencia de un ángulo θ alrededor del eje y para obtener el marco B .

- Rotación del marco de referencia de un ángulo ϕ alrededor del eje x para obtener el marco B .

El orden de estas rotaciones no es arbitrario, se llevan a cabo desde el marco de referencia N al marco B . La matriz que representa estas rotaciones se denota por $\mathbf{R}_n^b = \mathbf{R}_b^n$.

La matriz transpuesta implica que el mismo resultado se obtiene de transformar un vector del marco de referencia B al marco N , invirtiendo el orden de rotación. Esta secuencia de rotaciones es matemáticamente equivalente a:

$$\mathbf{R}_n^b = \mathbf{R}_{z,\psi} \mathbf{R}_{y,\theta} \mathbf{R}_{x,\phi} \quad (1)$$

Realizando esta operación se obtiene:

$$\mathbf{R}_b^n = \begin{bmatrix} c\psi c\theta & -s\psi c\phi + c\psi s\theta s\phi & s\psi s\phi + c\psi c\phi s\theta \\ s\psi c\theta & c\psi c\phi + s\phi s\theta s\psi & -c\psi s\phi + s\theta s\psi c\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix} \quad (2)$$

Una vez conocida la matriz de rotación, se pueden derivar las siguientes expresiones:

$$\phi = \text{atan2}(r_{32}, r_{33}) \quad (3)$$

$$\theta = -\sin^{-1}(r_{31}) = -\tan^{-1}\left(\frac{r_{31}}{\sqrt{1 - r_{31}^2}}\right) \quad (4)$$

$$\psi = \text{atan2}(r_{21}, r_{11}) \quad (5)$$

en donde r_{ij} denota el elemento (i, j) de la matriz de rotación $\mathbf{R}_b^n$, y $\text{atan2}(y, x)$ es la función arco tangente de dos argumentos.

En cuanto a la transformación de velocidades, que relaciona el vector de velocidad lineal en un marco de referencia inercial al marco de referencia del cuerpo, se puede resumir de la siguiente manera:

$$\dot{\boldsymbol{\eta}} = \mathbf{J}\mathbf{v} \quad (6)$$

$$\mathbf{J}(\mathbf{v}) = \begin{bmatrix} \mathbf{R}_b^n & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{T}_\Theta \end{bmatrix} \quad (7)$$

donde $\mathbf{T}_\Theta$ está dada por:

$$\mathbf{T}_\Theta = \begin{bmatrix} 1 & s\phi t\theta & c\phi t\theta \\ 0 & c\phi & -s\phi \\ 0 & \frac{s\phi}{c\theta} & \frac{c\phi}{c\theta} \end{bmatrix} \quad (8)$$

y

$$\dot{\boldsymbol{\eta}} = \frac{d(\boldsymbol{\eta})}{dt}. \quad (9)$$

2.2. Modelo Dinámico

La dinámica de un robot submarino sumergido se rige por las ecuaciones de movimiento de Newton-Euler extendidas para considerar el entorno marino. El modelo se expresa en su forma compacta como:

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{g}(\boldsymbol{\eta}) = \boldsymbol{\tau} + \mathbf{g}_0 + \mathbf{w}$$

Donde $\boldsymbol{\nu}$ y $\dot{\boldsymbol{\nu}}$ son la velocidad y la aceleración lineal y angular, respectivamente, medidas en el marco de referencia del cuerpo (*Body-Fixed*).

El modelo se compone de cuatro fuerzas y momentos principales que debe superar el sistema de control, los cuales se describen a continuación:

Componente	Símbolo	Descripción
Fuerzas Inerciales	$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu}$	Compuesta por dos términos: la matriz de inercia $\mathbf{M}$, que incluye la masa del cuerpo y la masa añadida (masa de agua desplazada que se mueve con el robot), y las fuerzas de Coriolis y Centrífugas $\mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu}$, que surgen del movimiento rotacional.
Fuerzas Hidrodinámicas	$\mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu}$	Representan el amortiguamiento o resistencia que el agua ejerce sobre el movimiento. Esto incluye las fuerzas viscosas y el arrastre (<i>drag</i>), que generalmente dependen del cuadrado de la velocidad del vehículo.
Fuerzas de Restitución	$\mathbf{g}(\boldsymbol{\eta})$	Son las fuerzas estáticas de gravedad $\mathbf{W}$ y flotación $\mathbf{B}$, dadas por el Principio de Arquímedes. Estas fuerzas buscan restaurar el robot a su condición de equilibrio.
Fuerzas de Control (Actuadores)	$\boldsymbol{\tau}$	Es el vector de fuerzas y momentos generados por los propulsores u otros actuadores, y es el término que utiliza el sistema de control para manipular el movimiento del robot.

Tabla 2: Componentes de las fuerzas del modelo dinámico

Adicionalmente, el modelo incluye las fuerzas de perturbación ambiental: $\mathbf{g}$ y $\mathbf{w}$ representan las perturbaciones externas o del entorno, como el impacto de las olas o las corrientes marinas sobre el vehículo.

2.3. Modelo Robot proporcionado

Para garantizar la fidelidad entre el comportamiento del robot en el simulador y el modelo matemático de control, los parámetros de la ecuación dinámica se han extraído directamente del archivo de descripción de escena `model.sdf`. A continuación, se detallan las matrices resultantes que configuran el modelo físico del vehículo.

2.3.1. Fuerzas Inerciales

La matriz de masa y momentos de inercia del cuerpo rígido ($\mathbf{M}_{RB}$) se construye a partir de la masa total del vehículo ($m = 10$ kg) y sus momentos principales de inercia. Considerando que el vehículo es simétrico en sus planos principales, los productos de inercia son despreciables, resultando en una matriz diagonal:

$$\mathbf{M}_{RB} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{I} \end{bmatrix} = \begin{bmatrix} 10 & 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0,0987 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0,1776 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0,2692 \end{bmatrix}$$

Donde la submatriz de inercia rotacional $\mathbf{I}$ contiene los valores $I_{xx} = 0,0987$, $I_{yy} = 0,1776$ y $I_{zz} = 0,2692$ $\text{kg} \cdot \text{m}^2$.

Adicionalmente, la matriz de fuerzas de Coriolis y centrípetas $\mathbf{C}_{RB}(\boldsymbol{\nu})$ se calcula en función de la velocidad del vehículo. Dados los valores de inercia anteriores, esta matriz toma la siguiente forma numérica dependiente del estado:

$$\mathbf{C}_{RB}(\boldsymbol{\nu}) = \begin{bmatrix} 0 & -10r & 10q & 0 & 10w & -10v \\ 10r & 0 & -10p & -10w & 0 & 10u \\ -10q & 10p & 0 & 10v & -10u & 0 \\ 0 & 10w & -10v & 0 & 0,269r & -0,177q \\ -10w & 0 & 10u & -0,269r & 0 & 0,098p \\ 10v & -10u & 0 & 0,177q & -0,098p & 0 \end{bmatrix}$$

2.3.2. Fuerzas Hidrodinámicas

El amortiguamiento hidrodinámico se modela mediante una matriz diagonal $\mathbf{D}(\boldsymbol{\nu})$ que incluye términos de arrastre cuadrático, característicos de cuerpos sumergidos. Los coeficientes d_i se han obtenido del simulador:

$$\mathbf{D}(\boldsymbol{\nu}) = -\text{diag}(d_1|u|, d_2|v|, d_3|w|, d_4|p|, d_5|q|, d_6|r|)$$

Sustituyendo los valores del modelo SDF:

$$\mathbf{D}(\boldsymbol{\nu}) = - \begin{bmatrix} 33,8|u| & 0 & 0 & 0 & 0 & 0 \\ 0 & 54,27|v| & 0 & 0 & 0 & 0 \\ 0 & 0 & 73,37|w| & 0 & 0 & 0 \\ 0 & 0 & 0 & 4,0|p| & 0 & 0 \\ 0 & 0 & 0 & 0 & 4,0|q| & 0 \\ 0 & 0 & 0 & 0 & 0 & 4,0|r| \end{bmatrix}$$

2.3.3. Fuerzas de Restitución

Las fuerzas gravitatorias y de flotación dependen de la posición del Centro de Gravedad (CoG) y del Centro de Flotación (CoB) respecto al origen del sistema de coordenadas del cuerpo.

- **Centro de Gravedad (CoG):** $[0, 0, 0,011]$ m
- **Centro de Flotación (CoB):** $[0, 0, 0,0325]$ m (deducido geoméricamente del volumen sumergido).

El vector de fuerzas de restitución $\mathbf{g}(\boldsymbol{\eta})$ se calcula considerando el peso $W = mg \approx 98,1$ N y el empuje $B = \rho g V \approx 98,4$ N. Debido a la pequeña distancia entre el CoG y el CoB, el vector resultante simplificado en posición de reposo presenta una ligera fuerza neta positiva hacia la superficie:

$$\mathbf{g}(\boldsymbol{\eta})_{\text{reposo}} \approx \begin{bmatrix} 0 \\ 0 \\ -0,3 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (\text{Unidades: N para fuerzas lineales, N} \cdot \text{m para pares})$$

2.3.4. Fuerzas de Control

El término $\boldsymbol{\tau}$ representa el vector de fuerzas y momentos generados por los propulsores. Para obtener este valor, el sistema de control procesa la diferencia entre el estado deseado y el estado actual mediante el cálculo del error y la ley de control utilizada, cuyo funcionamiento detallado se explica en los apartados posteriores.

3. Diseño del Sistema de Control

3.1. Desvinculación del AUV

Para poder trabajar con el modelo del robot limpio, sin que dependiera de ningún archivo previo vinculado a este, se han modificado ciertos programas del repositorio original.

En primer lugar, se ha desvinculado del archivo `model.sdf`, en el cual se conectaban sus motores directamente con el puente MAVROS. Se ha eliminado de dicho archivo también el plugin de `ArduPilot` que conectaba los motores y la IMU con el `ArduSub`.

Por otro lado, se ha modificado también el archivo con el cual se simula el mundo del Orca4, conocido como `sim.launch.py`. En este documento, se han declarado a `False` todos los argumentos que se conectarán con `ArduSub` y el puente MAVROS, así como comentado todos los procesos que involucran dichos términos.

3.2. Arquitectura del Nodo de Control

A la hora de extraer datos de los sensores implementados, simplemente ha sido necesario suscribirse a los nodos donde publican los resultados y hacer uso de los datos como entradas al controlador. Inicialmente también se hacía uso de la odometría para calcular la altura; sin embargo, se consiguió instalar un medidor de profundidad (barómetro/sensor de presión).

Como se puede observar en la imagen extraída del `rqt_graph` (Figura 1), se aprecia la estructura de comunicación entre nodos:

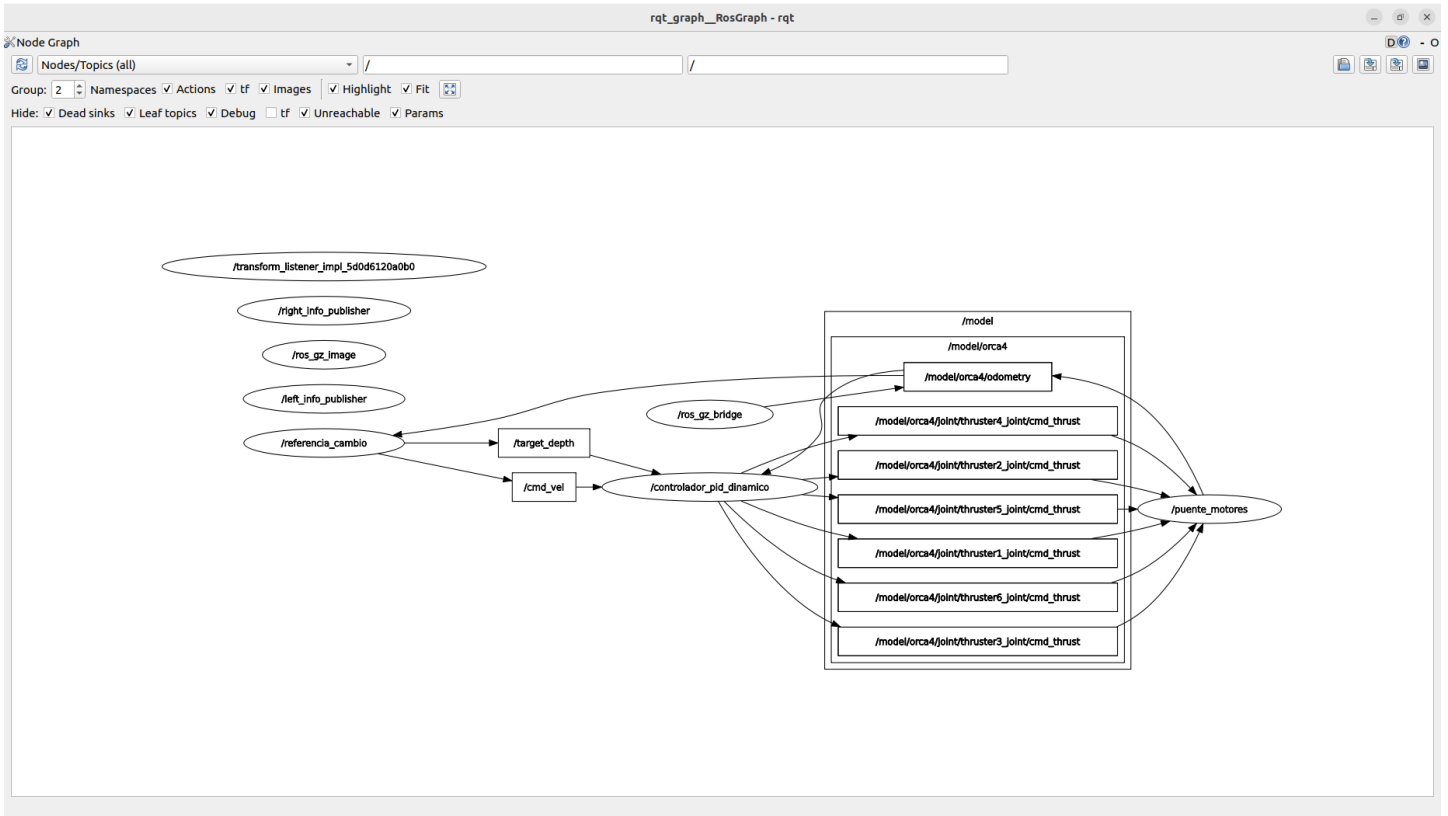


Figura 1: Arquitectura de nodos y tópicos del sistema (rqt_graph)

3.3. Estrategias de Control

Debido a que este proyecto se basaba en desarrollar un controlador de bajo nivel, se decidió de base intentar realizar varias estrategias y compararlas. Se tiene en cuenta que, al tratarse de un modelo de un robot físico, hay muchos términos despreciables que se tratarán según las necesidades de cada control.

Se pensó en tres estrategias de control, tal y como se comentó en la introducción:

- Control PD básico.
- Control PID basado en dinámica.
- Control SDRE (*State-Dependent Riccati Equation*).

Lo primero para el control es saber qué señales de control se están mandando a los motores y extraer los valores de los sensores necesarios para poder actuar conforme a sus medidas.

Todos los motores se controlan por fuerza en Newtons (N), cuyo rango operativo se define con un mínimo de -100 N y un máximo de $+100$ N. La señal de control tendrá valores normalizados entre -1 y 1 para simular un porcentaje del PWM (*Pulse Width Modulation*) en la realidad, ya que en el modelo idealmente no se hace uso de intensidades eléctricas.

$$f_i = u_i \cdot 100$$

Dado que el vector de empuje unitario en el marco de referencia del propio propulsor se define hacia abajo como $\mathbf{u}_{local} = [0, 0, -1]^T$, es necesario rotar este vector al marco del cuerpo utilizando la matriz de rotación basada en los ángulos de Euler (Roll ϕ , Pitch θ , Yaw ψ) de montaje de cada motor. La matriz de rotación general $\mathbf{R}$ empleada es:

$$\mathbf{R} = \begin{bmatrix} C\psi C\theta & C\psi S\theta S\phi - S\psi C\phi & C\psi S\theta C\phi + S\psi S\phi \\ S\psi C\theta & S\psi S\theta S\phi + C\psi C\phi & S\psi S\theta C\phi - C\psi S\phi \\ -S\theta & C\theta S\phi & C\theta C\phi \end{bmatrix}$$

Donde $C = \cos$ y $S = \sin$.

Una vez obtenido el vector de fuerza orientado en el marco del cuerpo ($\mathbf{f}_{body,i}$), se calculan las fuerzas totales (F_T) y los momentos totales (τ_T) sumando la contribución de los $N = 6$ motores:

$$\mathbf{F}_T = \sum_{i=1}^6 (f_i \cdot \mathbf{R}_i \cdot \mathbf{u}_{local})$$

$$\boldsymbol{\tau}_T = \sum_{i=1}^6 (\mathbf{r}_i - \mathbf{r}_{CG}) \times (f_i \cdot \mathbf{R}_i \cdot \mathbf{u}_{local})$$

Donde $\mathbf{r}_i$ es la posición del motor i y $\mathbf{r}_{CG} = [0, 0, 0, 011]$ es la posición del centro de gravedad.

Datos de los motores (extraídos de `model.sdf`):

Las poses de los motores (p_i) se definen como vectores de 6 elementos $[x, y, z, \phi, \theta, \psi]$. A continuación se detallan los parámetros geométricos utilizados para construir la matriz de asignación:

- **Motor 1:** $p_1 = (0, 14, -0,092, 0, -\frac{\pi}{2}, \frac{\pi}{2}, -\frac{\pi}{4})$
- **Motor 2:** $p_2 = (0, 14, 0,092, 0, -\frac{\pi}{2}, \frac{\pi}{2}, -\frac{3\pi}{4})$
- **Motor 3:** $p_3 = (0, 14, -0,092, 0, -\frac{\pi}{2}, \frac{\pi}{2}, \frac{\pi}{4})$
- **Motor 4:** $p_4 = (0, 14, 0,092, 0, -\frac{\pi}{2}, \frac{\pi}{2}, \frac{3\pi}{4})$

- **Motor 5 (Vertical):** $p_5 = (0, -0,109, 0,077, 0, 0, 0)$
- **Motor 6 (Vertical):** $p_6 = (0, 0,109, 0,077, 0, 0, 0)$

Esta configuración geométrica permite desacoplar los grados de libertad, utilizando los motores 1-4 para el movimiento en el plano horizontal (Avance, Desvío, Guiñada) y los motores 5-6 para el control de profundidad (Vertical) y cabeceo/alabeo si fuera necesario.

3.3.1. Cálculo de Fuerzas y Pares de Actuación

En función de las direcciones de actuación de los motores ($\mathbf{u}_i$), obtenemos las fuerzas y pares expresados en el Sistema de Referencia (SR) de la base del robot. Estas señales de control se escalan típicamente en un rango de -100 a 100 para su envío a los drivers de los motores.

El proceso fundamental consiste en invertir la matriz de configuración (Matriz de Asignación $\mathbf{T}$). En este caso, al tratarse de una matriz cuadrada (6 grados de libertad y 6 propulsores), es invertible, lo que permite calcular la fuerza necesaria que debe ejercer cada motor para lograr un par y fuerza vectorial total deseados.

Matemáticamente, si $\boldsymbol{\tau}$ es el vector de fuerzas y momentos generalizados requeridos por el controlador, y $\mathbf{u}$ es el vector de empujes de los motores individuales:

$$\mathbf{u} = \mathbf{T}^{-1} \boldsymbol{\tau}$$

O, en el caso generalizado para matrices no cuadradas, utilizando la pseudoinversa de Moore-Penrose ($\mathbf{T}^\dagger$):

$$\mathbf{u} = \mathbf{T}^\dagger \boldsymbol{\tau}$$

De este modo, conociendo la posición y dirección de actuación de cada motor, se extraen las fuerzas individuales que, sumadas vectorial y mecánicamente, generan la fuerza y el par neto sobre el cuerpo del robot. Esto permite gestionar el acoplamiento entre los movimientos de traslación y rotación, distribuyendo la potencia óptima a cada propulsor.

3.3.2. Cálculo del Error mediante Cuaterniones

Inicialmente, el cálculo de errores se planteó utilizando ángulos de Euler. Sin embargo, durante el desarrollo del proyecto se migró al uso de cuaterniones, dado que es el formato de datos extraído directamente de los sensores (IMU) y ofrece una representación más fiable y concisa, evitando problemas matemáticos como el bloqueo de cardán (*Gimbal Lock*).

Teniendo el cuaternión actual del robot con respecto al mundo ($\mathbf{q}_{actual}$) y el cuaternión deseado ($\mathbf{q}_{des}$), el cual generalmente representa una estabilización horizontal, el cuaternión de error ($\mathbf{q}_e$) se calcula mediante el producto del deseado por el conjugado (o inverso) del actual:

$$\mathbf{q}_e = \mathbf{q}_{des} \otimes \mathbf{q}_{actual}^{-1}$$

Donde el cuaternión resultante se expresa como $\mathbf{q}_e = (w, x, y, z)$. Para el controlador, el error de actitud real $\mathbf{e}$ se extrae considerando la parte vectorial ponderada por el signo de la parte escalar, asegurando que el robot tome el camino más corto en la rotación:

$$\mathbf{e} = \text{sgn}(w) \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

3.4. Estrategias de Control Implementadas

3.4.1. Control PD Básico

Una vez obtenido el error vectorial de actitud $\mathbf{e}$ descrito anteriormente, y disponiendo de las velocidades angulares $\boldsymbol{\omega}$ medidas por el giroscopio, se aplica una ley de control Proporcional-Derivativa (PD). Los pares de control necesarios ($\boldsymbol{\tau}$) se calculan como:

$$\boldsymbol{\tau} = K_p \cdot \mathbf{e} - K_d \cdot \boldsymbol{\omega}$$

Donde K_p y K_d son las matrices de ganancia proporcional y derivativa, respectivamente, que deben sintonizarse por asignación de polos sin cancelar dinámica. Finalmente, este vector se traduce a empujes de motor mediante la matriz de asignación mencionada previamente ($\mathbf{u} = \mathbf{T}^\dagger \boldsymbol{\tau}$).

Análisis del Controlador: Este controlador aporta inicialmente:

- **Robustez geométrica:** Al utilizar cuaterniones, el cálculo del error de actitud es globalmente estable y libre de singularidades.
- **Desacoplamiento de actuadores:** La matriz de asignación abstrae la complejidad mecánica, permitiendo al controlador “pensar” en fuerzas cartesianas (X, Y, Z) y pares (Roll, Pitch, Yaw).
- **Rapidez de respuesta:** La aplicación directa del PD genera respuestas ágiles en el medio fluido.

Sin embargo, aunque es una base sólida, este control ignora aspectos físicos cruciales como la hidrodinámica (arrastre y masa añadida), lo que genera un error en régimen permanente. Idealmente, se debería añadir un término integral para compensar factores no modelados como la flotabilidad imperfecta o fuerzas de Coriolis, aunque esto introduce riesgos de estabilidad que se discuten a continuación.

3.4.2. Control PID Basado en Dinámica

Esta estrategia parte de la base del modelo dinámico completo. Al poder aproximar el sistema a un modelo de segundo orden, se aplica un método de cancelación de dinámicas o “Par Calculado” considerando un esquema PID. Esto establece una relación directa entre los parámetros del controlador y los términos físicos del sistema (masa, inercia).

Para simplificar la implementación inicial, se despreciaron términos cuyo efecto se consideró secundario a bajas velocidades, específicamente las fuerzas de Coriolis y los términos de arrastre cuadrático.

Evolución a Control PD: Durante las pruebas experimentales, se descubrió que, al tratarse de un modelo fiable con ruido y pequeñas discrepancias físicas, el término integral acumulaba error debido a oscilaciones constantes. Esto provocaba que el sistema se descontrolara tras un periodo de operación (*Integral Windup*). Por esta razón, se decidió prescindir del término integral y trabajar con un esquema PD sintonizado dinámicamente.

Adicionalmente, se observó un desequilibrio en las magnitudes de control: las ganancias necesarias para el control de avance (Surge) resultaban muy altas en comparación con las de estabilización. Esto causaba que el avance acaparara toda la potencia de los motores, inhabilitando el control de actitud. Para solucionar esto, se realizó un escalado de ganancias, priorizando la estabilidad sobre la velocidad de avance.

3.4.3. Control SDRE (State-Dependent Riccati Equation)

El método de la Ecuación de Riccati Dependiente del Estado (SDRE) es una técnica avanzada de control no lineal. A diferencia de los controladores lineales clásicos (PID o LQR) diseñados para un punto de operación fijo, el SDRE ofrece una solución sistemática que funciona en todo el rango de operación del vehículo, adaptándose a los cambios dinámicos complejos del entorno submarino.

Principio de Pseudo-linealización: El núcleo del método es la factorización SDC (*State-Dependent Coefficient*). Partiendo de la dinámica no lineal general del robot:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$$

Donde $\mathbf{x}$ es el vector de estado (velocidades y posiciones) y $\mathbf{u}$ las fuerzas de control. El paso clave consiste en factorizar la función no lineal $\mathbf{f}(\mathbf{x})$ como el producto de una matriz dependiente del estado y el propio estado:

$$\mathbf{f}(\mathbf{x}) = \mathbf{A}(\mathbf{x})\mathbf{x}$$

Esto permite reescribir la dinámica en una forma que tiene estructura lineal, aunque sus matrices cambian instante a instante:

$$\dot{\mathbf{x}} = \mathbf{A}(\mathbf{x})\mathbf{x} + \mathbf{B}(\mathbf{x})\mathbf{u}$$

En nuestro caso, la matriz $\mathbf{A}(\mathbf{x})$ encapsula toda la información física cambiante del submarino. Específicamente, incluye las fuerzas de Coriolis y, crucialmente, el amortiguamiento hidrodinámico (Drag), el cual varía con el cuadrado de la velocidad y era ignorado en el control PD básico.

El control SDRE se plantea entonces como un problema de regulación óptima en cada paso de tiempo, buscando minimizar una función de coste J que equilibra la precisión del seguimiento de la trayectoria contra el consumo energético de los motores.

$$J = \int_0^\infty (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt$$

Donde:

- **Q** (Matriz de penalización del estado): Es una matriz diagonal de 6×6 que pondera la importancia de reducir el error en cada variable de estado. Valores altos en sus componentes obligan al robot a ser muy preciso y rápido en la corrección, aunque pueden inducir oscilaciones.
- **R** (Matriz de penalización del control): Es una matriz diagonal que pondera el coste energético o esfuerzo de los actuadores. Valores altos penalizan el uso excesivo de los motores, obligando al robot a realizar movimientos más suaves y eficientes energéticamente.

Algoritmo del Nodo de Control SDRE

El algoritmo implementado en el nodo de ROS 2 ejecuta el siguiente ciclo de control en cada paso de tiempo (Δt):

1. **Medición:** Se leen los sensores de a bordo (Odometría e IMU) para construir el vector de estado actual $\mathbf{x}_{actual}$.
2. **Actualización Dinámica (SDC):** Se recalculan las matrices $\mathbf{A}(\mathbf{x})$ y $\mathbf{B}(\mathbf{x})$ utilizando los valores instantáneos de velocidad. En este paso es donde el controlador “siente” los cambios físicos, como el aumento de la resistencia hidrodinámica cuando el vehículo acelera.
3. **Resolución de Riccati:** Se resuelve numéricamente la Ecuación Algebraica de Riccati (ARE) para el sistema linealizado en ese instante:

$$\mathbf{A}^T(\mathbf{x})\mathbf{P} + \mathbf{P}\mathbf{A}(\mathbf{x}) - \mathbf{P}\mathbf{B}(\mathbf{x})\mathbf{R}^{-1}\mathbf{B}^T(\mathbf{x})\mathbf{P} + \mathbf{Q} = \mathbf{0}$$

4. **Cálculo de Ganancia:** Se obtiene la matriz de retroalimentación óptima $\mathbf{K}(\mathbf{x})$:

$$\mathbf{K}(\mathbf{x}) = \mathbf{R}^{-1}\mathbf{B}^T(\mathbf{x})\mathbf{P}(\mathbf{x})$$

5. **Ley de Control:** Finalmente, se calcula y aplica el vector de fuerzas a los motores:

$$\mathbf{u} = -\mathbf{K}(\mathbf{x}) \cdot \mathbf{e}$$

La elección del control SDRE para este proyecto se justifica por su capacidad inherente para manejar las fuertes no-linealidades hidrodinámicas del entorno submarino. Al actualizar el modelo linealizado en cada iteración del bucle de control, el sistema ofrece una respuesta óptima tanto en maniobras de baja velocidad (donde predomina la inercia) como en desplazamientos rápidos (donde las fuerzas de arrastre dominan la dinámica).

4. Pruebas y Resultados

Para validar el sistema de control y la integración con el simulador, se diseñó un protocolo de pruebas progresivo. Inicialmente, hay que tener en cuenta que el AUV tiende a estabilizarse por sí solo debido a su diseño de flotabilidad pasiva. Por tanto, para evaluar la robustez del controlador, es necesario probarlo bajo condiciones de perturbación o induciendo señales de control constantes que lo desestabilicen. Debido a la falta de implementación de ruido ambiental (corrientes o “viento” submarino) en el simulador base, se usaron señales constantes de control que el sistema debía contrarrestar.

4.1. 1ª Prueba: Validación de Actuadores

La primera fase consistió en la validación individual de los actuadores. En un sistema de 6 grados de libertad, es crítico asegurar que el mapeo de software se corresponde exactamente con la ubicación física y el canal de comunicación del motor en el simulador. Esta prueba verifica que el flujo de datos desde ROS 2 llega correctamente a cada *thruster* sin interferencias cruzadas.

Metodología: Se desarrolló el nodo `mapeo.py`, que implementa una lógica secuencial activando cada motor durante 4 segundos con una fuerza de 200 N. Se configuró un perfil de QoS (*Reliable* y *Volatile*) para garantizar la llegada prioritaria de los comandos.

Resultados:

- **Identificación:** Los 6 motores respondieron en el orden secuencial correcto, validando el mapa de tópicos.
- **Estabilidad:** La respuesta fue lineal y estable ante la entrada escalón de fuerza.
- **Seguridad:** La función de parada de emergencia (`stop_all`) funcionó correctamente al interrumpir el proceso.

4.2. 2ª Prueba: Telemetría y Sensores

El objetivo fue verificar la recepción y transformación matemática de los datos sensoriales, pasando de formatos brutos (cuaterniones) a formatos de navegación humana (ángulos de Euler). Para ello se utilizó el nodo `angulos.py`.

Implementación: El nodo se suscribe a la odometría (`/model/orca4/odometry`) y al altímetro láser. Utiliza la librería `tf_transformations` para convertir la orientación en tiempo real.

Resultados:

- **Orientación:** Se visualizó correctamente la inclinación del submarino (Roll, Pitch, Yaw) en grados decimales.
- **Profundidad:** El sistema extrajo con precisión la cota Z y la distancia al fondo marino mediante el altímetro, incluyendo un filtrado de valores infinitos para seguridad.

4.3. 3ª Prueba: Cambio de Referencia Dinámico (Teleoperación)

Esta prueba implementó una interfaz de teleoperación mediante teclado (`cambio_ref.py`) para permitir que un operador envíe consignas de posición, orientación y velocidad en tiempo real.

Mapeo de Control: Se diseñó una interfaz multimodal que separa los comandos de velocidad directa de los *setpoints* de posición fija. El mapeo validado se muestra a continuación:

Categoría	Teclas	Acción	Incremento
Velocidad (X, Y)	w, s, a, d	Movimiento plano horizontal	0.2 m/s
Profundidad (Z)	i, k	Subir / Bajar (Heave)	0.25 m
Pitch (Cabeceo)	o, l	Inclinar morro arriba / abajo	5.0°
Roll (Alabeo)	u, y	Inclinación lateral izq / der	5.0°
Yaw (Giro)	j, ñ	Orientación brújula izq / der	5.0°
Reset	Espacio	Pone todas las referencias a 0	Instantáneo

Tabla 3: Mapeo de comandos de referencia para teleoperación

Conclusiones: El incremento escalonado de las referencias permitió realizar cambios suaves de trayectoria, evitando inestabilidades dinámicas. La sincronización entre los comandos enviados y la respuesta en `/target_orientation_euler` fue perfecta.

4.4. 4ª Prueba: Implementación de perturbaciones

En entornos submarinos reales, los vehículos están sometidos constantemente a corrientes marinas y fuerzas hidrodinámicas no modeladas. Por ello, esta prueba tiene como objetivo evaluar el comportamiento del controlador avanzado SDRE cuando se introducen fuerzas de perturbación constantes.

A diferencia de los controladores estándar, el nodo `control_sdre_perturbaciones` introduce una metodología específica para manejar el entorno hostil:

- **Inyección de Corriente Marina:** Se ha programado un vector de fuerza externa denominado $\mathbf{F}_{corriente}$. Este vector inyecta una perturbación constante definida como:
 - 1,0 N en el eje X .
 - -1,0 N en el eje Y .
 - 0,5 N en el eje Z .
- **Compensación mediante acción integral:** Para evitar que la corriente desvíe al robot permanentemente, el control SDRE que inicialmente no disponía de efecto integral se le incluye asignándole pesos distintos a los motores para que actúen en consecuencia. Esta componente acumula el error inducido por la corriente y genera una fuerza opuesta que permite al submarino “luchar” contra la perturbación hasta anular el error de posición en estado estacionario.
- **Adaptación Dinámica (SDRE):** El controlador recalcula las matrices de estado $\mathbf{A}(\mathbf{x})$ y $\mathbf{B}(\mathbf{x})$ en cada paso de tiempo. Al incluir el valor absoluto de la velocidad actual en el cálculo del amortiguamiento, el sistema ajusta su agresividad en función de si la corriente está acelerando o frenando al vehículo. Esto no afecta al término integral que es constante.

Resultados del Experimento

Los resultados de la prueba confirmaron la alta fidelidad del controlador SDRE frente a condiciones adversas:

Rechazo de Perturbaciones: Tras la inyección de la “corriente”, el submarino experimentó una desviación inicial mínima antes de que la acción integral estabilizara la trayectoria, recuperando la posición deseada en menos de t segundos con un error residual despreciable.

Estabilidad en 6-DOF: A pesar de las perturbaciones lineales, el acoplamiento dinámico fue gestionado eficientemente, manteniendo la orientación (roll, pitch, yaw) dentro de los márgenes de seguridad sin presentar oscilaciones divergentes.

4.5. 5ª Prueba: Implementación y Validación del Nodo de Guiado Autónomo

Esta nueva etapa de pruebas se alcanzó con la implementación del nodo de guiado autónomo (`control.punto.punto.py`). El objetivo principal fue validar la capacidad del robot para ejecutar una misión compleja de principio a fin sin intervención humana, gestionando de forma autónoma las transiciones entre inmersión, navegación y ascenso.

El nodo actúa como el cerebro de la misión, implementando una arquitectura de *Máquina de Estados Finita* que divide la operación en cuatro fases lógicas:

INMERSIÓN: El robot desciende verticalmente manteniendo su posición (X, Y) hasta alcanzar la profundidad operativa definida en el código ($z = -7,0$ m). La transición ocurre cuando el error de profundidad es menor a 0,5 m.

NAVEGACIÓN: El robot recorre secuencialmente la lista de *waypoints* preconfigurada: (10, 10), (20, 10), (20, 20) y (0, 0).

ASCENSO: Una vez completada la ruta, el sistema conmuta automáticamente a esta fase, ordenando el retorno a la superficie ($z = 0,0$ m).

FIN: Estado terminal donde se detiene la publicación de comandos de velocidad por seguridad.

Para la navegación horizontal, se implementó un algoritmo de seguimiento basado en un controlador proporcional (P). El nodo calcula el error euclídeo hacia el siguiente punto de paso y genera una velocidad proporcional a dicho error. Para garantizar la estabilidad dinámica del vehículo, se incluyó una saturación de la velocidad máxima a 1,5 m/s.

El criterio de conmutación entre puntos se define mediante un radio de aceptación de 1,0 m. Cuando la odometría indica que el submarino ha entrado en este radio, el nodo considera el objetivo como “alcanzado” e incrementa el índice del *waypoint*.

Resultados de la prueba

La misión se completó con éxito en el entorno de simulación, destacando los siguientes hitos:

- **Precisión en el Guiado:** El submarino visitó los cuatro puntos de la trayectoria con un error residual mínimo, validando el bucle de realimentación de odometría.
- **Gestión de Fases:** La transición de la fase de INMERSIÓN a NAVEGACIÓN fue fluida, activándose exactamente al confirmar la profundidad de $-7,0$ m.
- **Seguridad Automática:** Al alcanzar el último punto (origen), el robot ejecutó la maniobra de ascenso autónomo, finalizando la misión en superficie y deteniendo los propulsores.

5. Conclusiones

Tras la finalización del proyecto, se ha logrado con éxito la transición desde una plataforma dependiente de terceros (ArduSub y MAVROS) hacia un sistema de control de bajo nivel íntegramente desarrollado en ROS 2. Este proceso de desvinculación ha permitido una comprensión profunda de la cadena de mando, desde la recepción de telemetría hasta la publicación de fuerzas en los propulsores mediante el puente `ros_gz_bridge`.

En cuanto al rendimiento de los controladores:

PD básico: Se ha consolidado como una solución de control robusto gracias al uso de cuaterniones, evitando singularidades en la actitud. No obstante, al ser un controlador sin dinámica, ignora la física del robot (masa e hidrodinámica), lo que se traduce en un error en régimen permanente al no ser capaz de compensar por sí solo la flotabilidad positiva del AUV.

PID basado en dinámica: Aunque preciso en teoría, presentó dificultades en la práctica. Las altas ganancias requeridas para el desplazamiento traslacional saturaron los propulsores, dejando al sistema sin margen de maniobra para las correcciones de actitud. Esta falta de “autoridad de control” en los ángulos de inclinación provocó que el robot fuera incapaz de mantener la horizontalidad durante los desplazamientos, haciendo inviable este controlador sin un escalado de ganancias agresivo.

SDRE: Se posiciona como la variante más avanzada al ser capaz de modelar no-linealidades críticas como el amortiguamiento hidrodinámico. Además, la implementación del control SDRE permite una gestión inteligente de los recursos del AUV. Con visión de implementarlo en la vida real, el sistema no solo garantiza la estabilidad, sino que optimiza el consumo de batería y reduce el desgaste mecánico de los propulsores, adaptándose dinámicamente a las fuerzas de arrastre no lineales del entorno marino. Además, se ha realizado una implementación de término intergal para poder gestionar los errores en régimen permanente que se originan por las perturbaciones inducidas.

En cuanto a los problemas y limitaciones, los mayores retos técnicos fueron la compleja desvinculación del firmware original y la falta de documentación específica sobre robótica submarina, lo que obligó a una investigación profunda sobre el modelo dinámico del robot. En cuanto a las perturbaciones, fuimos incapaces de simular corrientes marinas en Gazebo; por tanto, se implementó un nodo externo de perturbaciones que permitió validar la robustez de los controladores en un entorno más realista.

A pesar de que el plazo de entrega y los conocimientos iniciales en hidrodinámica fueron limitantes, el uso de estrategias como el SDRE demostró ser la solución más eficaz, logrando gestionar las no-linealidades y optimizar el consumo energético frente a controladores clásicos.

Como línea de desarrollo futura, se propone la evolución del sistema de navegación actual. El controlador de alto nivel desarrollado se limita a un esquema de puntos de consigna (*point-to-point*). Se plantea sustituir este esquema por un planificador de trayectorias continuas, implementando generación de curvas suaves, optimización dinámica y evitación de obstáculos.

A. Guía de instalación de Orca4

A.1. Paso 1: Añadir repositorios adicionales (APT)

En primer lugar, se añaden los repositorios necesarios para la instalación de Gazebo Harmonic. Para ello, se comienza añadiendo la llave de seguridad de OSRF:

```
sudo apt update
sudo apt install curl gnupg lsb-release -y
sudo curl -fsSL https://packages.osrfoundation.org/gazebo.gpg \
-o /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
```

A continuación, se añade la dirección del repositorio de Gazebo al sistema:

```
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] \
http://packages.osrfoundation.org/gazebo/ubuntu-stable \
$(lsb_release -cs) main" | sudo tee \
/etc/apt/sources.list.d/gazebo-stable.list > /dev/null
```

Por último, se actualiza el gestor de paquetes e instala Gazebo Harmonic:

```
sudo apt update
sudo apt install gz-harmonic
```

A.2. Paso 2: Compilar el piloto automático (ArduSub SITL)

En este paso se compila el piloto automático ArduSub en modo SITL. Primero, se crea una carpeta donde se almacenará el proyecto y se clona el repositorio de ArduPilot:

```
mkdir cpr
cd ~/cpr
git clone https://github.com/ArduPilot/ardupilot.git
```

A continuación, se accede al repositorio y se realiza la compilación:

```
cd ardupilot
git submodule update --init --recursive
./waf configure --board sitl
./waf sub
```

Al finalizar el proceso, el ejecutable `ardusub` se encontrará en la siguiente ruta:

```
/home/usuario/cpr/ardupilot/build/sitl/bin/ardusub
```

A.3. Paso 3: Instalación de dependencias de compilación

Se instalan todas las dependencias necesarias para la correcta compilación y ejecución del entorno de simulación:

```
sudo apt install -y \
ros-humble-ros-gz-sim \
ros-humble-ros-gz-bridge \
python3-vcstool \
rapidjson-dev \
libprotobuf-dev \
protobuf-compiler \
libgstreamer1.0-dev \
libgstreamer-plugins-base1.0-dev \
libgz-sim7-dev \
```

```
libgz-transport12-dev \  
libgz-msgs9-dev \  
libgz-plugin-dev
```

A.4. Paso 4: Preparación del workspace (colcon_ws)

Se prepara el workspace de ROS 2 donde se compilará el proyecto Orca4. En primer lugar, se accede a la carpeta `src`:

```
cd ~/cpr/colcon_ws/src
```

Posteriormente, se importan las dependencias del proyecto:

```
vcs import < orca4/workspace.repos
```

A.5. Paso 5: Instalación de dependencias de ROS y MAVROS

Una vez preparado el workspace, se instalan las dependencias de ROS mediante `rosdep`:

```
cd ~/cpr/colcon_ws  
rosdep update  
rosdep install -y --from-paths src --ignore-src
```

A continuación, se instalan las dependencias necesarias para MAVROS:

```
wget https://raw.githubusercontent.com/mavlink/mavros/master/mavros/scripts/install_geographiclib_datasets.sh  
chmod a+x install_geographiclib_datasets.sh  
sudo ./install_geographiclib_datasets.sh
```

A.6. Paso 6: Compilación final

Con todas las dependencias instaladas, se procede a compilar el workspace completo:

```
cd ~/cpr/colcon_ws  
colcon build
```

A.7. Paso 7: Comprobación de la instalación

Para comprobar que la instalación se ha realizado correctamente, primero se añade el ejecutable `ardusub` al `PATH` del sistema:

```
export PATH=$PATH:/home/usuario/cpr/ardupilot/build/sitl/bin
```

Después, se carga el workspace de ROS 2:

```
source ~/cpr/colcon_ws/install/setup.bash
```

Finalmente, se lanza la simulación:

```
ros2 launch orca_bringup sim_launch.py
```

Si todos los pasos se han seguido correctamente, se abrirá la plataforma de simulación en Gazebo mostrando el robot submarino y el entorno acuático en el que se desplaza.

B. Guía de puesta en marcha

Esta guía describe el procedimiento para ejecutar, modificar y compilar los diferentes controladores desarrollados para el vehículo submarino en el entorno de simulación.

B.1. Ejecución del Entorno

Para lanzar el simulador y el sistema de control base en el mundo acuático, ejecute el siguiente comando en su terminal:

```
ros2 launch controlador_submarino controller.launch.py
```

B.2. Compilación y Modificaciones

Cada vez que se realice una modificación en el código fuente de los controladores o en los archivos de lanzamiento, es imperativo recompilar el *workspace*.

```
# Compilar el paquete especifico
colcon build --packages-select controlador_submarino
# Cargar las variables de entorno
source install/setup.bash
```

B.3. Selección del Controlador

Para cambiar el controlador activo, se debe editar el archivo `controller.launch.py` dentro de la carpeta `launch`.

B.3.1. Evolución de los controladores

Dependiendo de la prueba que se desee realizar, puede sustituir el ejecutable en el archivo *launch* por alguno de los siguientes:

Categoría	Módulos / Archivos	Funcionalidad, Entradas y Salidas
Pruebas de Base	angulos	Devuelve la profundidad y la orientación extraídas de la odometría.
	mapeo	Genera mensajes específicos para la prueba de canales de los motores.
	mover	Prueba de movimiento; envía mensajes directos para evaluar el comportamiento físico de los propulsores.
Gestión de Referencias	cambio_ref	Entradas: Comandos de teclado definidos en la tabla 3. Salidas: Referencias dinámicas del sistema en cada instante.
	cambio_pos	Transforma las referencias de consigna de cuaterniones a ángulos de Euler.
Desarrollo Evolutivo	controlador.py, controlador2.py, controlador_PD, controlador_PID	Primeras versiones de control cinemático (P/PD/-PID) sin considerar el modelo del robot.
	control_dinamica, control_din_PID, control_din_PID_odom	Evolución de controladores integrando el modelo dinámico inicialmente con odometría y posteriormente con sensores.
Versiones Finales	control_din_PID_odom.cambio	Implementación óptima del control dinámico PID con realimentación de odometría.
	control_sdre	Controlador óptimo no lineal basado en la ecuación de Riccati dependiente del estado sin odometría.

Tabla 4: Clasificación y Funcionalidad de los Módulos de Control Desarrollados

B.4. Gestión de Referencias y Alto Nivel

Para misiones complejas y cambios de consigna en tiempo real, utilice los siguientes scripts:

- **Control de Alto Nivel:** control_punto_punto (Navegación autónoma).
- **Referencias:** cambio_pos, cambio_ref y mover.

B.5. Pruebas de Robustez (Corrientes)

Para evaluar el comportamiento ante perturbaciones, el controlador más avanzado es el `control_sdre_perturbaciones`. Este nodo inyecta fuerzas externas constantes (corrientes) para evaluar la capacidad de rechazo del sistema mediante su acción integral.

Referencias

- [1] hama6767. Ros2 gazebo simulator for underwater robots, including robot model, dynamics, controller, rl stuffs., May 2025.
- [2] Musa Morena Marcusso Manhães, Sebastian A. Scherer, Martin Voss, Luiz Ricardo Douat, and Thomas Rauschenbach. UUV simulator: A gazebo-based package for underwater intervention and multi-robot simulation. In *OCEANS 2016 MTS/IEEE Monterey*. IEEE, sep 2016.
- [3] Patryk Cieślak. Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ros interface. In *OCEANS 2019 - Marseille*, jun 2019.
- [4] Evan Palmer, Christopher Holm, and Geoffrey Hollinger. Angler: An Autonomy Framework for Intervention Tasks with Lightweight Underwater Vehicle Manipulator Systems. In *IEEE International Conference on Robotics and Automation*, 2024.
- [5] Mabel M. Zhang, Woen-Sug Choi, Jessica Herman, Duane Davis, Carson Vogt, Michael McCarrin, Yadunund Vijay, Dharini Dutia, William Lew, Steven Peters, and Brian Bingham. Dave aquatic virtual environment: Toward a general underwater robotics simulator. In *Proceedings of the IEEE/OES Autonomous Underwater Vehicle (AUV) Symposium*, 2022.
- [6] LoicLiquid. Open source simulator for maritime robotics researchers, Dec 2020.
- [7] Clyde McQueen. orca4: ROS2 AUV based on the BlueROV2, ArduSub and Navigation2, June 2022.